

Retrieval-Augmented LLMs for Security Incident Analysis

Xavier Cadet
Dartmouth College
Hanover, NH, USA
xavier.fjf.cadet@dartmouth.edu

Aditya Vikram Singh
Northeastern University
Boston, MA, USA
singh.adityav@northeastern.edu

Harsh Mamania
Northeastern University
Boston, MA, USA
mamania.h@northeastern.edu

Edward Koh
Dartmouth College
Hanover, NH, USA
Edward.W.Koh@dartmouth.edu

Alex Fitts
PUNCH Cyber Analytics
Reston, VA, USA
alex@punchcyber.com

Dirk Van Bruggen
PUNCH Cyber Analytics
Reston, VA, USA
dirk@punchcyber.com

Simona Boboila
Northeastern University
Boston, MA, USA
m.boboila@northeastern.edu

Peter Chin
Dartmouth College
Hanover, NH, USA
Peter.Chin@dartmouth.edu

Alina Oprea
Northeastern University
Boston, MA, USA
a.oprea@northeastern.edu

Abstract

Investigating cybersecurity incidents requires collecting and analyzing evidence from multiple log sources, including intrusion detection alerts, network traffic records, and authentication events. This process is labor-intensive: analysts must sift through large volumes of data to identify relevant indicators and piece together what happened. We present a RAG-based system that performs security incident analysis through targeted query-based filtering and LLM semantic reasoning. The system uses a query library with associated MITRE ATT&CK techniques to extract indicators from raw logs, then retrieves relevant context to answer forensic questions and reconstruct attack sequences. We evaluate the system with eight LLM configurations on malware traffic incidents and a multi-stage Active Directory attack. We find that LLMs have different performance and tradeoffs, with Claude Sonnet 4 achieving 94% and DeepSeek V3 achieving 89% average recall across 17 malware scenarios, while DeepSeek costs 15× less than Claude per analysis, and locally-deployed Llama 3.1:70b achieves 81% recall at zero per-query cost. Attack step detection on the Active Directory scenario reaches 100% precision and up to 96% recall with an enumeration prompt. These results demonstrate that combining targeted query-based filtering with RAG-based retrieval—confirmed essential by ablation studies—enables accurate, cost-effective security analysis within LLM context limits.

CCS Concepts

• **Security and privacy** → **Network security**; • **Computing methodologies** → **Artificial intelligence**; • **Information systems** → **Language models**.

Keywords

network security, incident analysis, retrieval-augmented generation, large language models, log analysis

ACM Reference Format:

Xavier Cadet, Aditya Vikram Singh, Harsh Mamania, Edward Koh, Alex Fitts, Dirk Van Bruggen, Simona Boboila, Peter Chin, and Alina Oprea. 2026. Retrieval-Augmented LLMs for Security Incident Analysis. In *ACM Conference on AI and Agentic Systems (CAIS '26)*, May 26–29, 2026, San Jose, CA, USA. ACM, New York, NY, USA, 21 pages. <https://doi.org/10.1145/3786335.3813136>

1 Introduction

Security Operations Centers operate under increasing pressure: cyber threats grow in both volume and complexity, yet the analyst workforce has not kept pace. Modern Security Information and Event Management (SIEM) deployments routinely generate thousands of alerts per day, but empirical studies show analysts can only examine a small subset, resulting in fatigue and overlooked incidents [11, 20, 22]. Investigating a security incident (identifying compromised assets, tracing attacker actions, understanding intrusion mechanisms) demands synthesis of evidence spread across intrusion detection systems, network telemetry, and endpoint logs. This labor-intensive work strains limited analyst capacity, motivating research into intelligent systems that can accelerate incident analysis.

Current detection mechanisms depend heavily on signatures and static rules, which cannot generalize to previously unseen attack patterns. Sophisticated adversaries exploit this gap through techniques like misconfigured certificate template abuse (e.g., ESC1 [18]), Kerberos ticket forgery [16], and stepwise lateral movement through network trust relationships. Such attacks can progress over hours or days, depositing forensic artifacts across disparate log sources (Suricata alerts, Zeek connection records, Windows authentication events) where no individual entry reveals the complete attack chain. Recent machine learning approaches address alert triage and anomaly scoring [21, 24], but these methods yield classifications or rankings, not the narrative reconstruction that incident responders require.



This work is licensed under a Creative Commons Attribution 4.0 International License.
CAIS '26, San Jose, CA, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2415-2/26/05
<https://doi.org/10.1145/3786335.3813136>

Large Language Models (LLM) are well-positioned to address this problem: they can interpret unstructured text, integrate scattered facts into coherent narratives, and communicate findings in natural language [25]. Feeding raw security logs directly into an LLM, however, is impractical. A single incident may produce millions of log entries, vastly exceeding model context limits [4]. These logs mix genuine attack indicators with overwhelming volumes of routine activity (normal authentications, benign connections, spurious alerts), burying the attack signal. Additionally, incident response often demands rapid turnaround, while LLM inference remains comparatively slow. These challenges motivate the design of new AI-enabled security systems that extract relevant events from such large volumes of logs and generate incident reports interpretable by security analysts.

We present a Retrieval-Augmented Generation (RAG)-based system for security incident analysis that extracts relevant threat indicators by running targeted queries on security logs, mapping the attack indicators to the MITRE ATT&CK framework, and feeding the subsequent results to an LLM with context augmented by semantically similar events to generate interpretable reports of security incidents. The framework operates in two modes: forensic question-answering and attack reconstruction. In the first mode, analysts pose specific questions (identifying infected hosts, command-and-control (C2) servers, or initial compromise vectors) and receive evidence-backed reports with explainable answers. In the second mode, the system reconstructs attack sequences by associating distinct forensic events into causal narratives of attack progression over time. Central to our approach is a security-aware context construction pipeline. Rather than passing raw logs to the LLM, we extract through targeted queries Indicators Of Compromise (IOCs), compute aggregates of log data, and organize results into semantically meaningful chunks that preserve cross-event relationships. The RAG architecture is essential in extracting the most relevant information from multiple security logs for inclusion in the LLM context, enabling the model to generate interpretable incident reports for security analysts. Our general architecture supports multiple LLM models—both cloud APIs and locally deployed models—enabling organizations to balance performance, cost, and data sensitivity.

We make the following contributions:

Problem formulation for LLM-based incident analysis. We frame security incident analysis as two complementary tasks: (a) structured question-answering, where the system answers forensic questions about indicators, hosts, and attacker actions using retrieval-augmented context, and (b) attack sequence reconstruction that correlates discrete forensic events into causal narratives of attack progression, evaluated against reference timelines.

Security-aware RAG architecture with cross-event semantic reasoning. We design a retrieval pipeline tailored to security data: targeted queries extract indicators, aggregators compress thousands of events into patterns, and semantic chunking preserves cross-event relationships. This enables LLMs to identify attacks requiring correlation across discrete events, such as the detection of Active Directory certificate abuse by recognizing Subject/SAN mismatches across certificate request and issuance operations. This demonstrates contextual interpretation that distinguishes coordinated exploitation from individually legitimate activities.

LLM model evaluation with cost-effectiveness analysis. We evaluate eight LLM configurations spanning cloud APIs (Claude Sonnet 4, DeepSeek V3, GPT-4, GPT-4o, GPT-5-mini, GPT-5.2) and local deployment (Llama 3.1:70b, Cisco Foundation-Sec-8B) on malware traffic and Active Directory attack scenarios. The best-performing configurations (Claude Sonnet 4, DeepSeek V3) achieve 94% and 89% average recall respectively across 17 malware scenarios, while attack step detection on Active Directory scenarios reaches 100% precision and up to 96% recall with an enumeration prompt. DeepSeek achieves near-equivalent accuracy to Claude at 15× lower cost, making it the most cost-effective cloud model. Local deployment (Llama 3.1:70b) provides data privacy with acceptable accuracy (81% recall) for environments with strict data governance requirements. Notably, the security-specialized Cisco Foundation-Sec-8B model does not outperform general-purpose models at 71% recall, suggesting that reasoning capability matters more than domain-specific pretraining for complex forensic tasks. The system completes malware analysis in 1–2 minutes per incident with cloud models, demonstrating real-time feasibility for incident response. Beyond aggregate recall, we analyze cross-source reasoning using structural recall and corroboration behavior metrics, showing that cross-chunk synthesis ability—not deployment modality or domain specialization—is the primary driver of performance differences across model families.

Baseline comparisons and failure mode analysis. We compare RAG against no-RAG and Suricata-only baselines, showing that preprocessing and access to multiple log sources are both necessary: no-RAG misses all attack infrastructure due to context limits, while Suricata-only misses identity attribution because IDS data lacks authentication records. We characterize three disjoint failure mode categories (query coverage, retrieval and size limits, LLM reasoning errors), each with concrete remediation strategies, providing actionable guidance for practitioners.

To support reproducibility, we release our implementation, including the query library, RAG pipeline, evaluation scripts, and pre-computed LLM reports for all 17 malware scenarios.¹ The malware traffic datasets are publicly available [7, 8, 17].

2 Related Work

LLMs for Cybersecurity. Large language models have emerged as tools for cybersecurity automation, with recent surveys cataloging applications across vulnerability detection, malware analysis, threat intelligence, and incident response [25]. This body of work demonstrates both the potential and limitations of LLMs for security tasks.

Early efforts adapted pre-trained models to cybersecurity text. SecureBERT [1] introduced a domain-specific BERT model trained on cybersecurity corpora, demonstrating improved performance on tasks involving MITRE ATT&CK terminology. CySecBERT [2] applied domain-adaptive pre-training to a large-scale cybersecurity corpus to improve performance on cybersecurity NLP tasks such as Named Entity Recognition and Cyber Threat Intelligence. For threat taxonomy mapping, TRAM fine-tunes SciBERT to map reports to MITRE ATT&CK techniques [15]. These models provide strong semantic representations for classification and embedding tasks.

¹<https://github.com/neu-nds2/llm-sec-incident-analysis>

Log-based anomaly detection represents a key application area. LogLLM [10] combines BERT for semantic extraction with Llama for sequence classification on system log datasets. LogPrompt [14] explores zero-shot anomaly detection through prompt engineering. These approaches focus on system logs (e.g., HDFS, Linux) and binary anomaly labels rather than security-specific logs with multi-stage attack patterns.

Recent work explores Multi-agent LLM systems to cyber security tasks such as insider threat detection (Audit-LLM [21] demonstrates reduced false positives) while SOC-specific systems like CORTEX [24] propose specialized agents for alert triage. Work on CTI automation [23] develops agents that extract Indicators of Compromise from threat reports. These systems advance alert-level processing but focus on triage and summarization rather than answering specific questions about detected incidents. Examining operational use, Singh et al. analyzed queries from 45 SOC analysts, finding that LLMs function as cognitive aids that augment rather than replace expertise [20].

In contrast to prior work focusing on alert triage or classification, our work addresses end-to-end incident analysis formulated as question-answering over heterogeneous security logs, processing data from intrusion detection systems, network monitors, and authentication events.

Retrieval-Augmented Generation. Retrieval-Augmented Generation (RAG) addresses limitations of LLMs—hallucination, outdated knowledge, and opacity—by conditioning generation on external evidence retrieved at inference time [13]. RAG has become the dominant paradigm for knowledge-intensive tasks where accuracy and traceability are required. Surveys categorize RAG architectures into retriever-centric, generator-centric, and hybrid designs, with recent advances in adaptive retrieval, iterative refinement, and self-reflection [9, 19].

Recent work applies RAG to security tasks. CyberRAG [3] presents an agentic framework for cyber attack classification, using iterative retrieval over domain-specific knowledge bases to reduce false positives and generate explanations. AgCyRAG [12] combines knowledge graph retrieval with vector search for security log analysis, orchestrating specialized agents to integrate structured threat intelligence with security log data. Work on threat intelligence automation [23] uses RAG to extract Indicators of Compromise from CTI reports and generate regular expressions to assist security analysts in establishing SIEM correlation rules. GraphRAG-based approaches [5] leverage ontology-backed knowledge graphs constructed from network logs to support LLM-driven query generation for network security monitoring. These systems demonstrate RAG’s applicability to security data, primarily focusing on attack classification and threat intelligence extraction.

Our work applies RAG to a different task: answering structured security questions over heterogeneous log sources (Suricata alerts, Zeek connections, Windows events). This requires specialized chunking strategies tailored to security data and retrieval that supports reasoning about attack progression across multiple log sources.

3 Problem Formulation

We formalize security incident analysis as a question-answering task and define the evaluation metrics used throughout our experiments.

3.1 Security Incident Analysis as Question Answering

Let $I \in \mathcal{I}$ denote a **security incident**, characterized by a collection of security logs L_I (e.g., Suricata alerts, Zeek connection logs, Windows event logs). We formulate incident analysis as question-answering: given an incident’s logs, the system must answer forensic questions about the attack.

DEFINITION 3.1 (EVALUATION QUESTIONNAIRE). A **questionnaire** $Q \sim \mathcal{Q}(I)$ is a finite collection of question-answer pairs drawn from the space of possible questionnaires over incident I :

$$Q = \{(q_i, a_i)\}_{i=1}^M$$

where:

- $q_i \in Q$ is a natural language question
- a_i is the reference answer
- M is the number of questions in the questionnaire

Beyond forensic identification, we also evaluate attack reconstruction: deducing the attack path a malicious actor took over a given timeframe. For this task, we pose a compound question (“What did the attacker do? What defenses should be implemented?”) and evaluate both attack steps and defense recommendations using precision (fraction of reported items matching the reference) and recall (fraction of reference items correctly identified). This formulation unifies both modes within the question-answering framework, although attack reconstruction requires richer evaluation against known attack timelines (Section 7).

3.2 Security Analyzer

The Security Analyzer takes raw security logs as input and produces answers to security questions. We decompose it into two modular components that can be tuned and improved independently.

DEFINITION 3.2 (SECURITY ANALYZER). A **Security Analyzer** SA is a function:

$$SA : \mathcal{L} \times \mathcal{Q} \rightarrow \mathcal{R}$$

where $\mathcal{L}$ is the space of security logs, $\mathcal{Q}$ is the question space, and $\mathcal{R}$ is the response space. For an incident I with logs $L_I \in \mathcal{L}$ and questionnaire $Q = \{(q_i, a_i)\}_{i=1}^M$:

$$SA(L_I, Q) = \{r_i\}_{i=1}^M \quad \text{where } r_i = SA(L_I, q_i)$$

Security Context Extraction. ($SCE : \mathcal{L} \times \mathcal{Q} \rightarrow \mathcal{C}$): This component interfaces with the SIEM platform to retrieve logs and applies a library of predefined IOC queries $\mathcal{F} = \{f_1, \dots, f_K\}$ to extract security-relevant features such as Suricata alerts, suspicious certificates, high-volume connections, authentication anomalies. A RAG retrieval function then selects the top- k most relevant chunks for each question based on semantic similarity.

RAG-LLM Analysis. ($LLM : C \times Q \rightarrow \mathcal{R}$): Given the retrieved context and a question, the LLM produces a structured response with supporting evidence. This modularity allows each component to be tuned and improved independently: the query set F determines detection coverage, the embedding model controls retrieval quality, k governs context size, and the LLM affects reasoning accuracy.

4 System Architecture

Our architecture comprises two components: Security Context Extraction, which reduces raw logs to security-relevant information through targeted queries, and RAG-LLM Analysis, which retrieves relevant context and prompts an LLM to answer forensic questions. Figure 1 illustrates the overall architecture.

4.1 Design Principles

Five principles guided our architectural decisions.

1) Filter before analysis. LLMs are too slow to process large volumes of raw logs and perform poorly on noisy data. We first extract security-relevant information using targeted IOC queries, allowing the system to scale to tractable log volumes and generate interpretable reports.

2) Embed once, query many times. Extracted data is semantically embedded into a vector database and cached to disk. The index is built once per dataset and can be queried repeatedly with different questions, LLMs, or prompt strategies without reprocessing.

3) Periodic analysis. Rather than invoking the LLM on every event, the system performs analysis at defined intervals, batching related activity into coherent time windows.

4) Provider-agnostic design. LLM interactions are abstracted behind a common interface, enabling substitution of providers to balance cost, latency, and privacy requirements.

5) Extensible IOC query library. Detection queries are maintained as a separate library with associated MITRE ATT&CK techniques, allowing extension without modifying the core pipeline.

4.2 Security Context Extraction

The system integrates with existing network monitoring infrastructure through standard APIs. Our implementation uses Security Onion, an open-source platform that aggregates network traffic using Suricata and Zeek, communicating through Elasticsearch REST API calls. The architecture generalizes to any SIEM exposing a query API.

Security Context Extraction relies on a library of IOC extraction queries, each targeting a specific activity type. The AD attack library covers 9 query types mapping to MITRE ATT&CK techniques (certificate abuse, Kerberos ticket forgery, lateral movement, etc.); malware traffic scenarios use a separate set targeting network-level IOCs. The full query-to-technique mappings and scoping notes are given in Appendix J; full query definition examples appear in Appendix F.

The current query library was developed iteratively for the evaluated attack types, validating each query against known attack data; extending coverage to novel attack patterns requires adding corresponding queries, though the modular design facilitates this.

Each query pairs a base filter with aggregators that compress matching events into ranked patterns, exposing behavioral signals that individual log entries would not reveal. This design is an architectural choice: the aggregation step reduces thousands of events into compact, semantically rich summaries that preserve cross-event patterns required for attack identification. The extracted aggregations are then embedded and indexed for retrieval, as described in the following section.

4.3 RAG-LLM Analysis

The RAG-LLM Analysis component transforms extracted security information into actionable incident reports. Given a set of security questions, it retrieves relevant context from the embedded document store and prompts an LLM to produce evidence-based findings. This design decouples index construction from analysis, enabling rapid experimentation with different questions, providers, and prompt strategies without reprocessing the source data.

Data Processing and Embedding. The Security Context Extraction component produces structured output containing aggregations and log entries from the SIEM platform. Before embedding, this output is split into semantically meaningful chunks rather than arbitrary text segments. The data chunks consist primarily of aggregate results, which summarize patterns across many events.

Chunks are embedded using a pre-trained sentence embedding model and indexed in a vector store for fast similarity search.² The complete index—chunks, metadata, and embeddings—is serialized to disk, allowing the embedding phase to run once per dataset.

Retrieval and Prompt Construction. Given a security question, the system retrieves relevant chunks from the embedding index using semantic similarity search. The question is embedded using the same model as the indexed chunks, and nearest neighbors are retrieved based on vector similarity. This retrieval-augmented approach grounds LLM responses in actual log data, reducing hallucination and enabling evidence-based reasoning that analysts can verify.

The prompt establishes the LLM’s role as a cybersecurity analyst, presents the question, and provides brief network context with topology information (address ranges, domain name, key infrastructure). Retrieved chunks appear next, each tagged with its source file to support attribution. The prompt concludes with an instruction to answer based on the provided evidence or indicate when information is insufficient. We provide the prompt template in Appendix B and a sample network context in Appendix C.

Security Questions. The system supports different categories of security questions depending on the analysis goal (Appendix D). *Identification questions* focus on indicators of compromise: infected hosts, compromised accounts, malicious domains, and command-and-control servers. *Timeline questions* focus on attack progression: what steps did the attacker take, in what order, and when. *Remediation questions* focus on defensive actions: what steps should be taken to contain the incident and prevent further damage. All question types use the same retrieval and prompting pipeline, with the LLM adapting its response format based on the question framing.

²Our implementation uses all-mpnet-base-v2 and FAISS; other embedding models and vector stores could be substituted.

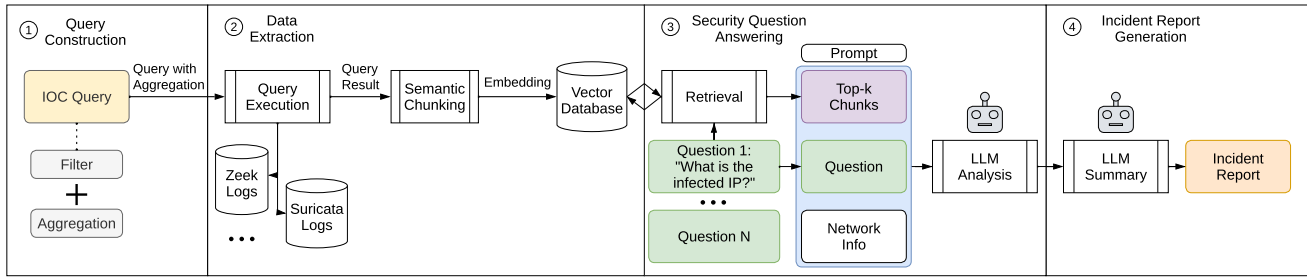


Figure 1: System architecture. Security Context Extraction queries the SIEM data, filtering the traffic to expose indicators of compromise. RAG-LLM Analysis embeds this data, retrieves relevant context for each question, and prompts an LLM to generate an incident report.

Multi-LLM Support. Following design principle number 4, the system abstracts LLM interactions behind a common interface, allowing different providers to be substituted without modifying the analysis pipeline. Currently supported providers include Anthropic, OpenAI, and DeepSeek as cloud APIs, and Ollama (Llama 3.1:70b) and Cisco as locally-deployed options.

Output Generation. The system produces a structured incident report containing findings for each security question, with responses that include specific evidence (IP addresses, hostnames, timestamps) and attribute findings to source files (Appendix E). A summary section follows, condensing the findings into a brief overview: attack type, key indicators, and recommended actions. The report concludes with metadata for reproducibility: the LLM provider, the number of chunks analyzed, the retrieval settings, and the timing information.

4.4 End-to-End Example

To illustrate how extraction and RAG-LLM analysis work together, consider a malware infection where the compromised host must be identified along with the responsible user account. The raw dataset contains 3,694 log events spanning HTTP connections, Kerberos authentication, IDS alerts, and TLS certificate exchanges (Fake Authenticator Malware scenario [8]).

The extraction queries reduce this to semantically rich summaries. The Suricata alert query identifies malicious activity:

Suricata High-Severity Alert Aggregation

```
{
  "high_severity_signatures": [
    {
      "key": "ET MALWARE Fake Microsoft Teams CnC",
      "doc_count": 2
    },
    {
      "key": "ET INFO PS1 Powershell File Request",
      "doc_count": 2
    }
  ],
  "high_severity_sources": [
    {
      "key": "10.1.17.215",
      "doc_count": 6
    }
  ]
}
```

These alerts confirm malicious activity from IP 10.1.17.215, but reveal neither the user responsible nor the specific payloads retrieved. The Kerberos client query provides identity attribution:

Kerberos Client Aggregation

```
{
  "key": "shutchenson",
  "doc_count": 11,
  "source_ips": [
    {
      "key": "10.1.17.215",
      "doc_count": 11
    }
  ]
}
```

The file download query reveals the attack payload:

File Download Aggregation

```
{
  "key": "10.1.17.215 -> 5.252.153.241:80 : /api/file/get-file /29842.ps1",
  "doc_count": 4
}
```

No single data source provides the complete picture. The IDS alerts lack user attribution; Kerberos logs show authentication but not malicious behavior; HTTP logs capture downloads but cannot distinguish malicious from benign traffic. When these aggregations are embedded and retrieved, the LLM corroborates across sources via the shared IP address, concluding that user shutchenson on workstation DESKTOP-L8C5GSJ downloaded a PowerShell payload from a command-and-control server.

This example illustrates how extraction reduces thousands of log events into three compact aggregations while preserving the semantic links (shared IP addresses) that enable cross-source reasoning. The complete queries and corresponding LLM output appear in Appendix F.

5 Experimental Overview

We evaluate the system on two complementary settings. The first is malware traffic analysis: 17 scenarios drawn from Malware-Traffic-Analysis.net, Unit 42 Wireshark Quizzes, and SANS ISC contests, evaluated as forensic question-answering (Section 6). The second is an Active Directory attack reconstruction with associated defense recommendation (Section 7).

Research Questions. We explore the following questions:

- RQ1 (Feasibility)** Can a RAG-based system achieve acceptable performance on security incident analysis?
- RQ2 (Model Comparison)** How do different LLM configurations compare in performance, latency, and cost?
- RQ3 (Context Sensitivity)** How does the number of retrieved chunks k affect performance?
- RQ4 (RAG Necessity)** Is RAG preprocessing essential, or can LLMs analyze raw security logs directly?
- RQ5 (Practical Utility)** Do system-generated defense recommendations align with expert-defined actions?

RQ1–RQ4 are addressed in the malware traffic evaluation (Section 6), which compares eight LLM configurations across 17 scenarios and evaluates a no-RAG baseline. RQ1 and RQ5 are addressed in the Active Directory evaluation (Section 7), which tests attack detection and defense recommendation on a multi-stage intrusion.

LLM Configurations. We evaluate eight configurations spanning cloud APIs and local deployment:

- **Claude Sonnet 4** (Anthropic): Cloud API
- **DeepSeek V3** (DeepSeek): Cloud API
- **GPT-4, GPT-4o, GPT-5-mini, GPT-5.2** (OpenAI): Cloud API
- **Llama 3.1:70b** (via Ollama): Local GPU deployment
- **Cisco Foundation-Sec-8B**: Local GPU deployment, security-specialized

RAG Configuration. The RAG pipeline uses the following parameters:

- **Embedding model:** all-mpnet-base-v2 (sentence transformer)
- **Vector store:** FAISS with cosine similarity
- **Retrieved chunks:** $k = 7$ per question (default); Section 6.3 examines sensitivity to k
- **LLM parameters:** temperature= 0.1, max output tokens= 4000

Each chunk contains a complete aggregation result rather than arbitrary text segments, preserving semantic coherence for retrieval.

Evaluation Metrics. We evaluate against reference sets derived from published scenario solutions and manual log analysis. Let R denote the reference set and A denote the model’s answer set. We compute:

$$\text{Precision} = \frac{|A \cap R|}{|A|}, \quad \text{Recall} = \frac{|A \cap R|}{|R|}$$

For single-value questions (e.g., infected host IP), precision equals recall and scoring is binary. For set-valued questions (e.g., C2 indicators), models receive partial credit proportional to the fraction of reference indicators found.

The malware evaluation primarily reports recall: reference answers capture *confirmed* malicious indicators from published solutions authored and reviewed by domain experts, not an exhaustive enumeration. Models may identify additional legitimately suspicious indicators absent from the reference set; Appendix H provides full precision-recall breakdowns. The AD evaluation reports both precision and recall, as its reference timeline was manually extracted through exhaustive log review of the Security Onion Kibana dashboard.

6 Malware Traffic Analysis

Evaluating LLM-based security analysis is constrained by the absence of public benchmarks pairing raw traffic with vetted forensic reference answers. To address this gap, we curated scenarios from three established sources whose solutions are authored and reviewed by domain experts: Unit 42 Wireshark Quizzes [7] (Palo Alto Networks’ threat research team), the SANS Internet Storm Center monthly forensic challenges [17] (with published official solutions), and Malware-Traffic-Analysis.net [8].

We evaluate our method on 17 scenarios (Table 8), drawn from these sources. Our evaluation spans 321 MB of PCAPs, 94,948 Security Onion log events, 129 forensic questions, 218 reference indicators.

6.1 Incident Analysis Performance

Table 1 presents recall for each model across 17 scenarios from three independent challenge sources, addressing **RQ1** (feasibility).

Table 1: Incident Analysis Recall (%) by Scenario and Model

Scenario	CL	Deep-Seek	LL	Cisco 8B	GPT 4	GPT 4o	GPT 5m	GPT 5.2
apr21isc	92	79	79	71	85	85	85	85
burnincandle	100	93	84	77	70	70	70	70
dec21isc	100	100	94	81	75	75	75	75
dirty_rat	89	75	71	43	50	64	50	50
easy_123	100	100	92	100	75	75	75	92
fake_auth	100	100	94	81	69	69	56	69
gozi	77	88	72	75	72	70	70	70
iced_id	94	91	88	75	50	50	38	50
jun21isc	98	80	70	38	54	60	54	54
koi_stealer	100	88	69	88	56	69	69	69
lumma_stealer	100	100	93	86	71	71	71	71
may21isc	86	86	79	79	79	79	64	64
net_rat	100	93	86	57	79	71	57	71
oct21isc	88	78	72	48	46	58	46	58
qakbot	90	78	84	69	69	62	69	62
redline_stlr	94	94	81	56	81	81	75	81
warmcookie	96	93	79	86	68	68	68	68
Average	94	89	81	71	67	69	64	68

CL=Claude Sonnet 4; LL=Llama 3.1:70b

Claude Sonnet 4 achieves the highest average recall (94%), followed by DeepSeek V3 (89%) and Llama 3.1:70b (81%). Locally-deployed models perform comparably to several cloud-based models: Llama (81%) and Cisco Foundation-Sec-8B (71%) exceed or match most GPT variants (64–69%), suggesting that organizations with data sensitivity requirements need not sacrifice accuracy.

The results reveal distinct capability differences across model families. GPT models consistently struggle with questions requiring cross-source integration, such as identifying C2 servers from behavioral aggregations (connection counts, file download volumes) correlated with Suricata alerts. Claude and DeepSeek achieve near-perfect recall on these questions, while GPTs often default to listing indicators from a single data source.

IcedID and the multi-host scenarios (oct21isc, jun21isc) prove most challenging overall. IcedID involves a multi-stage infection chain requiring correlation of redirect IPs, download URLs, and C2 domains across separate log sources (details in Appendix I). The multi-host scenarios (3 infected machines, 5+ malware families) increase the data volume and require the model to attribute indicators to specific hosts.

Finding 1: Recall is driven by a model’s cross-source reasoning ability, not deployment modality: locally-hosted models match or exceed cloud GPT variants. Multi-stage infections and multi-host scenarios prove most challenging, requiring cross-source correlation.

6.2 Reasoning and Cross-Source Integration

Beyond recall, models exhibit qualitatively different reasoning strategies, most evident on questions that structurally require cross-source synthesis. Forty-two of the 129 questions are set-valued with reference indicators distributed across two or more retrieval chunks—no single chunk contains the complete answer. We measure cross-source integration with two complementary metrics: *structural recall* (recall on the multi-chunk subset) and *corroboration behavior* (the fraction of set-valued answers whose self-reported chunk citations span two or more chunks).

Table 2: Cross-source integration on set-valued questions.

LLM	Recall on set-valued questions, hard (%)	Multi-chunk cite rate, soft (%)
Claude Sonnet 4	89	95
DeepSeek V3	78	88
Llama 3.1:70b	56	76
Cisco 8B	46	36
GPT-4	52	38
GPT-4o	53	31
GPT-5-mini	47	33
GPT-5.2	52	33

Claude reaches 89% recall on the multi-chunk subset and cites two or more chunks for 95% of set-valued answers; GPT-family models recall 47–53% and cite multi-chunk evidence for only 31–38%. The context-size ablation (Section 6.3) corroborates this: going from $k=1$ to $k=3$, Claude/DeepSeek gain 33–36 points while GPTs gain only 13–18, consistent with stronger cross-chunk synthesis in the former.

The reasoning gap is also visible qualitatively. In Appendix G, we compare responses to the C2 detection question (Q5) on the Koi Stealer scenario. DeepSeek identifies only the confirmed C2 server, explicitly rejecting domain-resolved IPs as insufficiently corroborated. Cisco produces verbose sequential reasoning, narrating its inspection of each data source before arriving at the same single correct answer as DeepSeek—but cites only one chunk rather than cross-validating, relying on the strongest single signal. GPT-5.2 lists several IPs without prioritization, including known Microsoft and Azure infrastructure as C2 candidates, treating the question as a data extraction task rather than an analytical one.

These patterns reflect a precision-coverage trade-off: DeepSeek minimizes false positives through evidence-weighted reasoning; Cisco reaches the same conservative answer through verbose sequential analysis but without cross-source synthesis; GPT-5.2’s undifferentiated enumeration—including known cloud provider addresses as C2 candidates—increases analyst burden. For production deployment, evidence-weighted reasoning (distinguishing confirmed threats from suspicious activity) is preferable to exhaustive listing. Appendix H provides detailed precision-recall breakdowns per scenario.

6.3 Impact of Context Size

We investigate how the number of retrieved chunks k affects performance (RQ3). We evaluate $k \in \{1, 3, 5, 7, 14\}$ across all 17 scenarios and 8 LLMs.

Table 3: Context-Size Ablation: Recall (%) by k

LLM	$k=1$	$k=3$	$k=5$	$k=7$	$k=14$
Claude Sonnet 4	39	72	86	94	94
DeepSeek V3	35	71	80	89	89
Llama 3.1:70b	40	68	78	81	78
Cisco 8B	43	62	72	71	67
GPT-4	46	60	64	67	66
GPT-4o	46	59	65	69	64
GPT-5-mini	44	62	65	64	66
GPT-5.2	46	62	64	68	64

At $k=1$, GPTs (46%) outperform Claude (39%) and DeepSeek (35%): Claude tends to answer “Not found” when only one chunk is available, while GPTs extract answers from whatever single chunk they receive. By $k=3$ this reverses as Claude integrates across chunks (72% vs 61% for GPTs), and the gap widens to 27 points at $k=7$ (94% vs 67%). This crossover reveals a differential cross-chunk integration ability: Claude and DeepSeek gain 33–36 points going from $k=1$ to $k=3$, while GPTs gain only 13–18.

At $k=14$, recall plateaus or slightly declines for most LLMs (Cisco −4, Llama −3, GPT-4o −5). On the largest scenario (jun21isc, 3 infected hosts), $k=14$ exceeds GPT context limits entirely. $k=7$ is a sweet spot across all LLMs.

Finding 2: LLMs differ in how much they benefit from additional context. The retrieval window k interacts with model reasoning: more context helps only if the model synthesizes across chunks. $k=7$ is a sweet spot; $k=14$ provides no further benefit.

6.4 Baseline Comparisons

No-RAG Baseline. We evaluate a no-RAG baseline to assess whether RAG preprocessing is essential or whether LLMs can analyze raw security logs directly (RQ4). For each security question, we send as many raw logs as fit within the model’s context window, without any feature extraction or embedding. This represents the simplest possible approach: direct LLM analysis of unprocessed security data. While a sliding window approach processing logs in multiple batches could eventually cover all data, it would require ~ 24 batches for this scenario’s 3,694 logs (Claude’s 158-log window), resulting in $24 \times 8 = 192$ API calls versus 8 for RAG.

Table 4: RAG vs No-RAG Baseline: Recall (%) on the fake_auth scenario

LLM	RAG ($k=7$)	No-RAG	Logs Seen by LLM
Claude Sonnet 4	100	50	158 (4.3%)
DeepSeek V3	100	50	104 (2.8%)
GPT-4	69	38	22 (0.6%)
GPT-4o	69	38	22 (0.6%)
GPT-5-mini	56	38	22 (0.6%)
GPT-5.2	69	38	22 (0.6%)

Table 4 compares RAG and no-RAG performance on the Fake Authenticator scenario. Context window constraints limit no-RAG approaches to 0.6–4.3% of available logs (22–158 out of 3,694). The logs that fit are chronologically first and contain primarily benign startup activity. All models correctly identify victim information (IP, hostname, timestamp) present in these early logs but miss all attack infrastructure: malicious domains, C2 servers, and suspicious certificates appear only in later logs that exceeded context limits.

Table 5: Timing vs Cost per LLM

LLM	Recall (%)	Avg time/ scenario	Avg time/ question	In (\$/M)	Out (\$/M)	Cost/ Anlys.
Claude Sonnet 4	94	100s	13.1s	3.00	15.00	\$0.12
DeepSeek V3	89	110s	14.4s	0.28	0.42	\$0.008
GPT-4	67	18s	2.4s	3.00 [†]	12.00 [†]	\$0.11 [†]
GPT-4o	69	19s	2.6s	2.50	10.00	\$0.09
GPT-5-mini	64	18s	2.4s	0.25	2.00	\$0.01
GPT-5.2	68	17s	2.3s	1.75	14.00	\$0.09
Llama 3.1:70b	81	200s	26.4s	—	—	\$0*
Cisco 8B	71	376s	49.5s	—	—	\$0*

[†]GPT-4.1 pricing used; GPT-4 pricing no longer published.

*Local deployment; requires GPU infrastructure, cost not included.

GPT models additionally confuse the Windows machine hostname with the human user account, a reasoning flaw that persists in both RAG and no-RAG settings.

Suricata-Only Baseline. We also evaluated whether traditional IDS alerts alone suffice. Using only Suricata alerts from the Fake Authenticator scenario, an analyst could identify the infected IP and C2 infrastructure. However, hostname, user account, and initial infection domain are absent from IDS data entirely—these require Kerberos authentication logs and HTTP connection records. This yields 40% recall, comparable to no-RAG but with a different failure mode: no-RAG fails due to context limits, while Suricata-only fails due to missing data sources.

Finding 3: Both no-RAG and Suricata-only baselines achieve low recall but for different reasons: no-RAG misses attack infrastructure due to context limits; Suricata-only misses identity attribution due to lacking cross-source data. The performance gap from RAG demonstrates that both preprocessing and access to multiple log sources are essential.

6.5 Cost Analysis

We compare cost-effectiveness across models based on observed token usage. Table 5 summarizes pricing (from vendor documentation as of early 2026), analysis time, and performance. The cost per analysis is computed as:

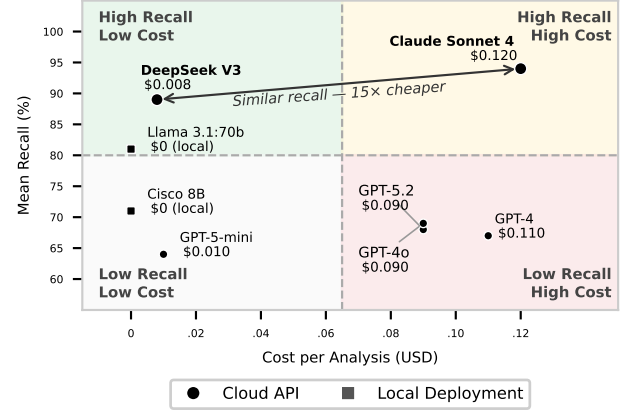
$$C_{\text{analysis}} = N_{\text{calls}} \cdot \left(\frac{T_{\text{in}}}{10^6} \cdot P_{\text{in}} + \frac{T_{\text{out}}}{10^6} \cdot P_{\text{out}} \right) \quad (1)$$

The number of LLM calls per analysis is $N_{\text{calls}} = 8$ (one per question). We use conservative estimates of input and output tokens per call: $T_{\text{in}} = 3000$ and $T_{\text{out}} = 400$. P_{in} , P_{out} are the model’s prices in USD per million tokens from Table 5.

DeepSeek achieves 89% recall at the lowest cost (\$0.008 per analysis), making it 15× cheaper than Claude while within 5 points of its recall. At operational scale (1,000 analyses/day), this translates to \$240/month versus \$3,600/month. Local deployment eliminates per-query costs but requires GPU infrastructure and incurs longer processing times: Llama (70B) achieves 81% recall at 3.3 min/analysis, while Cisco 8B achieves 71% at 6.3 min/analysis.

7 Active Directory Attack Analysis

Data. The attack data originates from a controlled red team exercise against an Active Directory environment. The scenario simulates a

**Figure 2: Cost vs. Recall Across LLM models.**

sophisticated attacker (Red) attempting to compromise a Windows domain while defenders (Blue) respond in real-time.

Red Team Actions: Initial compromise via Metasploit, certificate request with Administrator SAN, certificate enumeration, Kerberos TGT and service ticket requests, PSEXEC lateral movement to Domain Controller, malicious user creation, and ultimately SSH access to SCADA with service disruption (attack objective).

Blue Team Actions: Certificate revocation, KRBTGT reset, Domain Controller restart, password reset, and user account disable. With defensive response, Blue successfully prevented SCADA compromise.

Evaluation. Unlike the malware scenarios with discrete question-answer pairs, the AD evaluation uses the attack recreation task: given a timeframe, identify attack steps and recommend defensive actions. We evaluate on 5-minute sliding windows (15:30–15:35, 15:35–15:40, 15:40–15:46), simulating real-time analysis (window size justification in Appendix K). The reference attack steps in the timeline were manually extracted through exhaustive log review via the Security Onion Kibana dashboard, enabling evaluation of both precision and recall. Each data filtering query produces a compact result fitting in a single chunk; with 9 queries generating 9 chunks, all relevant context is retrieved simultaneously. We prompt the LLM with an *enumeration prompt* that instructs it to list each attack step as a separate, timestamped entry—as opposed to a narrative summary—so that individual steps can be matched against the reference timeline.

7.1 Attack Step Detection

We evaluate attack-step detection across 8 LLMs (RQ1). Recall ranges from 46% to 96%, with Claude Sonnet 4 and DeepSeek V3 reaching 96% (Table 6). The performance gap is concentrated in the 15:30–35 window, where 11 distinct attack steps occur within five minutes: Claude and DeepSeek enumerate all 11, while GPT models report 2–5, leaning on Suricata alert labels and under-representing certificate, Kerberos, and service-installation evidence. The single missed step at 96% is a query-coverage gap in the kerberos_auth filter (see Section 8). Precision is 100% throughout—every reported attack step corresponded to actual malicious activity. The system

Table 6: AD attack reconstruction - steps correctly identified per window. Per-timestamp recall counts and overall average across LLMs. R = reference step count.

LLMs	15:30–35 (R = 11)	15:35–40 (R = 8)	15:40–46 (R = 7)	Avg Recall (%)
Claude Sonnet 4	11	8	6	96
DeepSeek V3	11	8	6	96
Llama 3.1:70b	9	6	6	81
Cisco 8B	8	7	6	81
GPT-4o	5	7	6	69
GPT-4	3	7	6	62
GPT-5.2	3	7	5	58
GPT-5-mini	2	6	4	46

correctly identified initial access (Metasploit, PowerShell injection, file drops), the credential theft chain (certificate enumeration, request, issuance, Kerberos tickets), and lateral movement (PSEXEC via service installation). Notably, recall ordering across LLMs on the AD scenario mirrors the malware evaluation, suggesting model selection generalizes across task types.

7.2 Multi-Step Attack Reconstruction

We examine how the system reconstructed an ESC1 certificate template abuse attack across five discrete events: certificate enumeration, certificate request with SAN:Administrator, certificate issuance with Subject:CN=<user>, Kerberos TGT as Administrator, and service ticket for krbtgt. Each event was captured by an independent query.

Given these events as separate chunks, the system produced:

Red enumerated certificates, then requested a certificate with SAN=Administrator. The certificate was issued, allowing impersonation of the Administrator account. Red then obtained a Kerberos TGT as Administrator and requested a service ticket for krbtgt, achieving domain-wide access.

The exploit mechanism required connecting information across chunks. The certificate *request* chunk contained SAN:upn=Administrator, while the certificate *issued* chunk showed Subject:CN=<user>. The LLM identified this Subject/SAN mismatch as the privilege escalation vector, then linked it to subsequent Kerberos events.

Finding 4: The system distinguishes attack from benign activity by recognizing that individually legitimate operations (certificate issuance, Kerberos authentication) constitute exploitation when they enable privilege escalation, demonstrating contextual interpretation beyond pattern matching.

7.3 Defense Recommendation Quality

We assess whether LLM-generated recommendations align with successful defensive actions (RQ5). Table 7 summarizes results for Claude Sonnet 4; DeepSeek V3 produces identical recommendations in windows 1–2 and omits RC in window 3.

Two semantic equivalences are applied when scoring. First, recommendations to isolate or quarantine the domain controller are counted as RD, since taking the DC offline achieves the same containment goal. Second, RP is counted as covered when DU is already recommended for the same compromised account: a disabled account cannot authenticate regardless of its password.

Table 7: Defense Recommendations (Claude Sonnet 4).

Window	Blue	LLM	P	R
15:30–35	RC, RK, RD	RC, RK, RD, DU, RP	3/5	3/3
15:35–40	RK, RD, RP, DU	DU, RK, RD	3/3	3/3
15:40–46	RD, RP, DU, RK	RC, DU, RD	2/3	2/3

RC=Revoke Cert, RK=Reset KRBTGT, RD=Restart DC, DU=Disable User, RP=Reset Password. DU covers RP.

Claude Sonnet 4 achieves 89% average recall and 76% average precision without being given the action vocabulary. Recommendations were contextually grounded: certificate revocation after detecting fraudulent certificate issuance, KRBTGT reset after Kerberos ticket abuse, user disable after unauthorized account creation.

7.4 Operational Feasibility

Per-LLM analysis latencies on the AD task are comparable to the per-question latencies reported in Table 5, as each window requires a single LLM call; the slowest model (Cisco 8B, under 2 minutes per window) still completes well within the 5-minute window.

This AD scenario represents a fast-moving attack with core exploitation compressed into 60 seconds. The 5-minute window suited this pace, but attack timescales vary widely: ransomware may complete encryption in under 5 minutes, while APT campaigns spread lateral movement over days. Window size must be tuned to threat characteristics. More fundamentally, analysis latency should be measured against response latency rather than attack speed. Defensive actions may require several minutes to execute, and coordinated responses involve human decision-making and approval workflows that typically dominate total incident response time. The system’s sub-minute analysis latency makes detection a small fraction of end-to-end response.

8 Failure Modes

Recall misses fall into three disjoint categories.

Query coverage. A reference indicator is not extracted from logs by any feature query. Fixes are local to the query library: widen filters or increase aggregation depth. For example, Elasticsearch aggregations return the top- N items per field; on *gozi*, $N=10$ silently dropped low-ranked indicators including the 11th C2 IP. Raising N closes the gap.

Retrieval and size limits. The relevant chunk does not reach the prompt, or the response is truncated. Fixes tune the retrieval window k , the prompt character limit, and the output token budget. Raising k from 3 to 7 raises Claude’s recall on *dec21isc* from 50% to 100%; raising the prompt character limit from 20 K to 60 K raises *jun21isc* from 27% to 98%; raising the output budget from 2 k to 4 k tokens removes truncations.

LLM reasoning. The chunk reaches the prompt but the model misinterprets it, fails to integrate across chunks, or formats the output incorrectly. Fixes are question rewording and model selection. On *qakbot*, GPT-4 returns the machine account desktop-e7fhjs4\$ instead of the user damon.bauer; appending “(not machine accounts ending in \$)” to the question resolves the confusion. On the AD scenario, an enumeration prompt (“list each discrete attack step with its timestamp”) replaces a narrative prompt and lifts Claude/DeepSeek to 96% recall.

The first two categories are architectural and affect all models uniformly; the third drives the LLM ranking.

9 Conclusion

We presented a retrieval-augmented generation system for security incident analysis that combines targeted query-based filtering with LLM semantic reasoning. Evaluation across eight LLM configurations on malware and Active Directory scenarios demonstrates that RAG preprocessing is essential (no-RAG baselines miss all attack infrastructure), LLM selection determines the performance ceiling once retrieval is properly configured, and DeepSeek V3 achieves near-equivalent accuracy to Claude Sonnet 4 (89% vs. 94% recall) at 15× lower cost. Notably, security-specialized models (Cisco Foundation-Sec-8B) do not outperform general-purpose LLMs, suggesting that reasoning capability matters more than domain-specific pretraining for forensic analysis. Our system's key capability is cross-event semantic reasoning: distinguishing coordinated exploitation from individually legitimate operations by correlating evidence across discrete forensic events.

Acknowledgments

This research was funded by the Defense Advanced Research Projects Agency (DARPA), under contract W912CG23C0031.

References

- [1] Ehsan Aghaei, Xi Niu, Waseem Shadid, and Ehab Al-Shaer. 2023. SecureBERT: A Domain-Specific Language Model for Cybersecurity. In *Security and Privacy in Communication Networks*, Fengjun Li, Kaitai Liang, Zhiqiang Lin, and Sokratis K. Katsikas (Eds.). Springer Nature Switzerland, Cham, 39–56. doi:10.1007/978-3-031-25538-0_3
- [2] Markus Bayer, Philipp Kuehn, Ramin Shanehsaz, and Christian Reuter. 2024. CySecBERT: A Domain-Adapted Language Model for the Cybersecurity Domain. *ACM Trans. Priv. Secur.* 27, 2 (April 2024), 18:1–18:20. doi:10.1145/3652594
- [3] Francesco Blefari, Cristian Cosentino, Francesco Aurelio Pironi, Angelo Furfaro, and Fabrizio Marozzo. 2026. CyberRAG: An Agentic RAG Cyber Attack Classification and Reporting Tool. *Future Generation Computer Systems* 176 (March 2026), 108186. doi:10.1016/j.future.2025.108186
- [4] Peng Cai, Reza Ryan, and Nickson M. Karie. 2025. LLMLogAnalyzer: A Clustering-Based Log Analysis Chatbot Using Large Language Models. arXiv:2510.24031 [cs] doi:10.48550/arXiv.2510.24031 arXiv:2510.24031.
- [5] Marco Carvalho, Fitzroy Nembhard, and Dhanish Mehta. 2025. Towards the Application of GraphRAG to Network Security. *The International FLAIRS Conference Proceedings* 38 (May 2025). doi:10.32473/flairs.38.1.138895
- [6] Brad Duncan. 2023. Cold as Ice: Unit 42 Wireshark Quiz for IcedID. <https://unit42.paloaltonetworks.com/wireshark-quiz-icedid/>. Accessed: 2026-02-25.
- [7] Brad Duncan. 2023. Unit 42 Wireshark Quiz Series (Gozi, Qakbot, RedLine Stealer, IcedID). <https://unit42.paloaltonetworks.com/category/cybersecurity-tutorials/>. Accessed: 2026-02-25.
- [8] Brad Duncan. 2026. Training Exercises - Malware-Traffic-Analysis.net. <https://www.malware-traffic-analysis.net/training-exercises.html>. Accessed: 2026-02-25.
- [9] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2024. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997 [cs] doi:10.48550/arXiv.2312.10997 arXiv:2312.10997.
- [10] Wei Guan, Jian Cao, Shiyu Qian, Jianqi Gao, and Chun Ouyang. 2025. LogLLM: Log-based Anomaly Detection Using Large Language Models. arXiv:2411.08561 [cs] doi:10.48550/arXiv.2411.08561 arXiv:2411.08561.
- [11] Leon Kersten, Tom Mulders, Emmanuele Zambon, Chris Snijders, and Luca Allodi. 2023. 'Give Me Structure': Synthesis and Evaluation of a (Network) Threat Analysis Process Supporting Tier 1 Investigations in a Security Operation Center. 97–111. <https://www.usenix.org/conference/soups2023/presentation/kersten>
- [12] Kabul Kurniawan, Rayhan Firdaus Ardian, Elmar Kiesling, and Andreas Ekelhart. 2025. AgCyRAG: An Agentic Knowledge Graph Based RAG Framework for Automated Security Analysis. In *Proceedings of the Second International Workshop on Retrieval-Augmented Generation Enabled by Knowledge Graphs (RAGE-KG 2025) ((CEUR Workshop Proceedings))*.
- [13] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., 9459–9474.
- [14] Yilun Liu, Shimin Tao, Weibin Meng, Feiyu Yao, Xiaofeng Zhao, and Hao Yang. 2024. LogPrompt: Prompt Engineering Towards Zero-Shot and Interpretable Log Analysis. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion '24)*. Association for Computing Machinery, New York, NY, USA, 364–365. doi:10.1145/3639478.3643108
- [15] MITRE. 2023. Threat Report ATT&CK Mapper (TRAM). <https://ctid.mitre.org/projects/threat-report-attck-mapper-tram/>.
- [16] MITRE ATT&CK. 2025. *Steal or Forge Kerberos Tickets (Technique T1558)*. <https://attack.mitre.org/techniques/T1558/>. Accessed: February 6, 2026.
- [17] SANS Internet Storm Center. 2021. SANS ISC Forensic Contest Diaries. <https://isc.sans.edu/diary/28160>. Series of monthly forensic contests. Accessed: 2026-02-25.
- [18] Will Schroeder and Lee Christensen. 2021. *Certified Pre-Owned: Abusing Active Directory Certificate Services*. Technical Report. SpecterOps. https://specterops.io/wp-content/uploads/sites/3/2022/06/Certified_Pre-Owned.pdf Whitepaper.
- [19] Chaitanya Sharma. 2025. Retrieval-Augmented Generation: A Comprehensive Survey of Architectures, Enhancements, and Robustness Frontiers. *arXiv preprint arXiv:2506.00054* (2025). <https://arxiv.org/abs/2506.00054>
- [20] Ronal Singh, Shahroz Tariq, Fatemeh Jalalvand, Mohan Baruwal Chhetri, Surya Nepal, Cecile Paris, and Martin Lochner. 2025. LLMs in the SOC: An Empirical Study of Human-AI Collaboration in Security Operations Centres. arXiv:2508.18947 [cs] doi:10.48550/arXiv.2508.18947 arXiv:2508.18947.
- [21] Chengyu Song, Linru Ma, Jianming Zheng, Jinzhi Liao, Hongyu Kuang, and Lin Yang. 2024. Audit-LLM: Multi-Agent Collaboration for Log-based Insider Threat Detection. arXiv:2408.08902 [cs] doi:10.48550/arXiv.2408.08902 arXiv:2408.08902.
- [22] Shahroz Tariq, Mohan Baruwal Chhetri, Surya Nepal, and Cecile Paris. 2025. Alert Fatigue in Security Operations Centres: Research Challenges and Opportunities. *ACM Comput. Surv.* 57, 9 (April 2025), 224:1–224:38. doi:10.1145/3723158
- [23] PeiYu Tseng, ZihDwo Yeh, Xushu Dai, and Peng Liu. 2024. Using LLMs to Automate Threat Intelligence Analysis Workflows in Security Operation Centers. arXiv:2407.13093 [cs] doi:10.48550/arXiv.2407.13093 arXiv:2407.13093.
- [24] Bowen Wei, Yuan Shen Tay, Howard Liu, Jinhao Pan, Kun Luo, Ziwei Zhu, and Chris Jordan. 2025. CORTEX: Collaborative LLM Agents for High-Stakes Alert Triage. arXiv:2510.00311 [cs] doi:10.48550/arXiv.2510.00311 arXiv:2510.00311.
- [25] Hanxiang Xu, Shenao Wang, Ningke Li, Kailong Wang, Yanjie Zhao, Kai Chen, Ting Yu, Yang Liu, and Haoyu Wang. 2025. Large Language Models for Cyber Security: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* (Sept. 2025). doi:10.1145/3769676

Appendix: Table of contents

A	Information about the Malware Scenarios	11
B	Prompt Template	11
C	Sample Network Context	12
D	Questions used for LLM Analysis	12
E	Sample LLM Answers	13
E.1	Identifying the IP of the potentially infected host	13
E.2	Identifying C2 infrastructure	14
F	Aggregation Queries	15
F.1	Kerberos Client Aggregation	15
F.2	Suricata High-Severity Alert Aggregation	15
F.3	File Download Aggregation	16
F.4	LLM Analysis Output	17
G	Comparative Analysis of LLM Reasoning	18
G.1	DeepSeek V3: Precision-Focused Analysis	18
G.2	Cisco Foundation-Sec-8B: Verbose Chain-of-Thought	19
G.3	GPT-5.2: Enumerative Listing	19
H	Precision-Recall Analysis	20
I	IcedID Dataset Complexity	20
J	MITRE ATT&CK Technique Coverage	21
K	Choice of Analysis Window	21

A Information about the Malware Scenarios

File sizes are obtained via capinfos. The number of Security Onion logs is obtained from the Kibana Dashboard associated with Security Onion. Scenarios are sourced from SANS ISC, Palo Alto Unit 42, and Malware-Traffic-Analysis.net.

Table 8: Evaluated Malware Scenario Dataset Statistics

Dataset	Source	Size	No. of Logs
jun21isc	ISC	43.1 MB	47,991
oct21isc	ISC	46.7 MB	7,278
apr21isc	ISC	12.9 MB	5,699
dec21isc	ISC	31.4 MB	3,869
may21isc	ISC	17.9 MB	1,544
iced_id	Unit-42	22.3 MB	5,701
qakbot	Unit-42	30.5 MB	4,279
gozi	Unit-42	5.67 MB	897
redline_stlr	Unit-42	1.35 MB	336
lumma_stealer	MTA	25.9 MB	4,435
fake_auth	MTA	25.5 MB	3,694
easy_123	MTA	6.27 MB	3,164
warmcookie	MTA	11.4 MB	2,385
burnincandle	MTA	6.53 MB	1,114
net_rat	MTA	20.2 MB	1,044
koi_stealer	MTA	2.03 MB	774
dirty_rat	MTA	10.9 MB	744
Total		321 MB	94,948

ISC: SANS ISC; Unit-42: Palo Alto Unit 42; MTA: Malware-Traffic-Analysis.net

B Prompt Template

Listing 1 shows the full prompt template sent to the LLM for each forensic question. The prompt is structured into five components. First, the agent is assigned the role of a cybersecurity analyst and provided with the specific forensic question to answer. Second, the agent is instructed to ground its response in specific evidence (IPs, timestamps, hostnames) or explicitly state when the provided data is insufficient. Third, the agent is required to conclude with a structured output: a `FINAL ANSWER` field containing the answer, and a `CITED CHUNKS` field listing the filenames of every retrieved chunk that materially supported the answer, enforcing traceability between the response and the underlying security logs. Fourth, the `{network_context}` field supplies the network topology information (appendix C), providing the agent with environmental context such as the LAN segment, domain, and active directory configuration of the monitored network. Fifth, the

Security Data section contains the top- k chunks retrieved from the vector index, each tagged with its source filename in brackets; these are the chunks the agent is expected to cite.

Listing 1: Full prompt template for RAG-based security analysis

```
You are a cybersecurity analyst. Analyze the security data and answer this question:

{question}

Answer with specific evidence (IPs, timestamps, hostnames) or state "Not found in provided data" if insufficient.
Present your analysis and then conclude with EXACTLY two lines, in this order:

FINAL ANSWER = [A, B, C, ...]
CITED CHUNKS = [filename1, filename2, ...]

CITED CHUNKS must list every [filename] block from the Security Data below that
materially supported your FINAL ANSWER. Use the exact filenames shown in
brackets at the start of each chunk (e.g. suricata_alerts_result.json).
Omit chunks you read but did not use. Do not invent filenames.

Network: {network_context}

Security Data:
[chunk_1_filename.json] {chunk_1_content}

[chunk_2_filename.json] {chunk_2_content}

...

[chunk_k_filename.json] {chunk_k_content}

Answer:
```

C Sample Network Context

Listing 2 is an example of network information provided to the LLM to help it understand the network environment. This information can be found as part of the problem description on the website hosting the PCAP files [6, 8], and generally includes the LAN, domain, and active directory domain controller.

Listing 2: Network topology information for the Fake Authenticator malware traffic scenario

```
- LAN segment range: 10.1.17.0/24 (10.1.17.0 through 10.1.17.255)
- Domain: bluemoontuesday.com
- Active Directory domain controller: 10.1.17.2 - WIN-GSH54QLW48D
- AD environment name: BLUEMOONTUESDAY
- LAN segment gateway: 10.1.17.1
- LAN segment broadcast address: 10.1.17.255
```

D Questions used for LLM Analysis

Questions are tailored per scenario; the full per-scenario questionnaires are available in the released code. Listing 3 shows the core questions asked across all malware scenarios, representative additional questions used in most scenarios, and the enumeration prompt used for the Active Directory attack reconstruction.

Listing 3: Representative forensic questions for malware traffic analysis and the enumeration prompt for Active Directory attack reconstruction

```
# === Core questions (all malware scenarios) ===
What is the IP address of the potentially infected internal host in the LAN?
What is the hostname of the potentially infected machine in the LAN?
What is the Windows human user account name (not machine accounts ending in $) of the potentially infected machine in the LAN?
```

```

What are the likely fake or suspicious domains / URLs for initial infection?
What are the suspicious external IP addresses contacted, which might be involved in command-and-control (C2) communication?

# === Additional questions (most malware scenarios) ===
During what hour did the first signs of infection activity appear? Provide the earliest date and time in YYYY-MM-DD HH:MM
UTC format.
According to the connection volume statistics, what single external IP address received the most connections from the
infected hosts? Is it a C2 server or a legitimate service?
Were any suspicious certificates observed in the encrypted network traffic? If so, identify the domain or IP in the
certificate.

# === Active Directory attack reconstruction ===
List each discrete attack step observed in this timeframe, including its timestamp and the specific action taken. Include
both new actions and any continuing activity from prior events. For each attack step, recommend a specific defensive
action. Enumerate every instance separately, even if the same technique is used multiple times.

```

E Sample LLM Answers

We present sample answers generated by Anthropic on the Fake Authenticator scenario, using a 7-chunks context and the Claude Sonnet 4 model.

E.1 Identifying the IP of the potentially infected host

Question: What is the IP address of the potentially infected internal host in the LAN?

The LLM Analyzer correctly identified the IP 10.1.17.215 as potentially infected, synthesizing evidence across four distinct data sources. It flagged suspicious domain communications (16 domains contacted solely by this host), high-severity Suricata alerts including Fake Microsoft Teams C2 signatures and PowerShell file requests, Kerberos authentication activity linking the IP to user account shutchenson and hostname DESKTOP-L8C5GSJ, and network context confirming the host's position within the internal LAN segment.

The model concludes with a structured FINAL ANSWER = [10.1.17.215], providing a machine-parseable answer directly usable for downstream processing. The CITED CHUNKS field lists the four source files that materially supported the answer: suspicious_domains_result.json, suricata_alerts_result.json, kerberos_hostnames_result.json, and kerberos_clients_result.json. Of the seven chunks retrieved by the RAG pipeline, the model selectively cited only those directly relevant to the final answer, omitting suspicious_certificates_result.json as unused, demonstrating selective, evidence-grounded attribution rather than blanket citation of all retrieved context (Listing 4).

Listing 4: Sample LLM answer to the question: What is the infected IP?

```

1. What is the IP address of the potentially infected internal host in the LAN?

Looking at the security data, I need to identify the potentially infected internal host by analyzing the various security
indicators.

Analysis:
IP Address 10.1.17.215 shows multiple concerning indicators:

1. Suspicious Domain Communications: This IP is the sole source for connections to 16 different suspicious domains including:
    - "appointedtimeagriculture.com" (suspicious domain name)
    - "authenticatoor.org" (typosquatting-like domain)
    - "google-authenticator.burleson-appliance.net" (suspicious subdomain)
    - "bttrack.com" (potentially BitTorrent related)

2. High-Severity Security Alerts: 10.1.17.215 appears as the top source IP (6 occurrences) in high-severity Suricata alerts,
including:
    - "ET MALWARE Fake Microsoft Teams CnC Payload Request (GET)" (2 occurrences)
    - "ET INFO PS1 Powershell File Request" (2 occurrences)
    - "ET INFO TeamViewer Dyngate User-Agent" (2 occurrences)

3. Kerberos Authentication Activity: The IP is associated with:
    - User account "shutchenson" (11 authentication events)
    - Computer accounts "DESKTOP-L8C5GSJ$" and "desktop-l8c5gsj$" (14 total events)

```

```

- Hostname "desktop-l8c5gsj"

4. Network Context: 10.1.17.215 is within the internal LAN segment (10.1.17.0/24) and shows extensive outbound communication
   patterns consistent with potential malware behavior.

The combination of malware-related alerts, suspicious domain communications, PowerShell file requests, and potential C&C
communications strongly indicates this host is compromised.

FINAL ANSWER = [10.1.17.215]

CITED CHUNKS = [suspicious_domains_result.json, suricata_alerts_result.json, kerberos_hostnames_result.json,
kerberos_clients_result.json]

RETRIEVED CHUNKS = [suspicious_domains_result.json, suspicious_certificates_result.json, kerberos_hostnames_result.json,
suspicious_certificates_result.json, suricata_alerts_result.json, suricata_alerts_result.json, kerberos_clients_result.
json]

```

E.2 Identifying C2 infrastructure

Question: What are the suspicious external IP addresses contacted, which might be involved in command-and-control (C2) communication?

The LLM Analyzer correctly identified four suspicious external IPs involved in C2 communication: 5.252.153.241, 185.188.32.26, 45.125.66.32, and 45.125.66.252. The model applied a confidence-tiered reasoning approach, distinguishing high-confidence C2 IPs confirmed by Spamhaus DROP blacklist hits and high-severity Suricata alerts from medium-confidence IPs flagged by suspicious certificate characteristics (IP-based and self-signed certificates). This tiered analysis demonstrates evidence-weighted reasoning, prioritizing corroborated indicators over individually suspicious signals. The CITED CHUNKS field correctly attributes the answer to two source files: suspicious_certificates_result.json and suricata_alerts_result.json, selectively omitting the remaining five retrieved chunks as unused (Listing 5).

Listing 5: Sample LLM Answer to the question: What is the C2?

```

5. What are the suspicious external IP addresses contacted, which might be involved in command-and-control (C2)
   communication?

Looking at the security data, I can identify several suspicious external IP addresses that are likely involved in command-
and-control (C2) communication based on multiple indicators:

Analysis

High-Confidence C2 IPs:
1. 5.252.153.241 - This IP appears 4 times in high-severity alerts and is flagged by "ET DROP Spamhaus DROP Listed Traffic
   Inbound group 5", indicating it's on a known malicious IP blacklist.
2. 185.188.32.26 - This IP appears 2 times in high-severity alerts and is also flagged by the same Spamhaus DROP alert,
   confirming malicious activity.

Medium-Confidence Suspicious IPs:
3. 45.125.66.32 and 45.125.66.252 - These IPs are using suspicious IP-based certificates (rather than domain-based
   certificates) and self-signed certificates, which are common C2 infrastructure characteristics. They also appear as
   sources in high-severity alerts.

Additional Context:
- The alerts include "ET MALWARE Fake Microsoft Teams CnC Payload Request (GET)" which directly indicates C2 communication
  attempts
- "ET INFO PS1 Powershell File Request" suggests potential malicious PowerShell activity
- All suspicious activity appears to involve internal host 10.1.17.215 (desktop-l8c5gsj) as either source or destination

The combination of Spamhaus blacklist hits, malware-specific signatures, and suspicious certificate usage provides strong
evidence these IPs are involved in C2 operations.

FINAL ANSWER = [5.252.153.241, 185.188.32.26, 45.125.66.32, 45.125.66.252]

CITED CHUNKS = [suspicious_certificates_result.json, suricata_alerts_result.json]

```

```
RETRIEVED CHUNKS = [suspicious_domains_result.json, suspicious_certificates_result.json, suspicious_certificates_result.json,
    suricata_alerts_result.json, suspicious_certificates_result.json, suricata_alerts_result.json,
    kerberos_hostnames_result.json]
```

F Aggregation Queries

The following Elasticsearch queries produce the aggregated results shown in Section 4.4. Each query pairs a filter with aggregations that compress matching events into ranked summaries.

F.1 Kerberos Client Aggregation

This query extracts usernames from Kerberos authentication events (port 88) and groups them by source IP address.

Listing 6: Kerberos Client Query

```
{
  "query": {
    "bool": {
      "must": [
        {"term": {"destination.port": 88}},
        {"range": {"@timestamp": {"gte": "...", "lte": "..."}}}
      ]
    }
  },
  "aggs": {
    "client_accounts": {
      "terms": {
        "script": {
          "source": "def msg = doc['message.keyword'].value;
            def pattern = /\\"client\\"\\.\\\"([^\\"\\\\/]+)\\/;/;
            def matcher = pattern.matcher(msg);
            if (matcher.find()) return matcher.group(1);"
        },
        "size": 20
      },
      "aggs": {
        "source_ips": {
          "terms": {"field": "source.ip", "size": 10}
        }
      }
    }
  },
  "size": 0
}
```

F.2 Suricata High-Severity Alert Aggregation

This query filters for IDS alerts from internal hosts and aggregates by signature name, source IP, and destination IP, with a nested filter for high-severity events.

Listing 7: Suricata Alerts Query

```
{
  "query": {
    "bool": {
      "must": [
        {"bool": {"should": [
          {"term": {"event.dataset": "suricata.alert"}},
          {"term": {"event.module": "suricata"}}
        ]}},
        {"range": {"source.ip": {"gte": "10.0.0.0", "lte": "172.31.255.255"}}}
      ]
    }
  }
```

```

    }
  },
  "aggs": {
    "high_severity_alerts": {
      "filter": {"range": {"rule.severity": {"gte": 1, "lte": 2}}},
      "aggs": {
        "high_severity_signatures": {
          "terms": {"field": "rule.name", "size": 10}
        },
        "high_severity_sources": {
          "terms": {"field": "source.ip", "size": 10}
        },
        "high_severity_destinations": {
          "terms": {"field": "destination.ip", "size": 5}
        }
      }
    }
  },
  "size": 0
}

```

F.3 File Download Aggregation

This query identifies file downloads by filtering for common download ports and file extensions, then uses a script to reconstruct HTTP request patterns from log messages.

Listing 8: File Download Query

```

{
  "query": {
    "bool": {
      "must": [
        {"terms": {"destination.port": [80, 443, 8080, 8443]}}
      ],
      "should": [
        {"wildcard": {"message": "*.ps1*"}},
        {"wildcard": {"message": "*.exe*"}},
        {"wildcard": {"http.uri": "*.ps1*"}},
        {"wildcard": {"http.uri": "*.exe*"}}
      ]
    }
  },
  "aggs": {
    "http_requests": {
      "terms": {
        "script": {
          "source": "def msg = doc['message.keyword'].value;
                    def pattern = /GET[ ]+([^\s]+)/;
                    def matcher = pattern.matcher(msg);
                    if (matcher.find()) {
                      return doc['source.ip'].value + ' -> ' +
                             doc['destination.ip'].value + ':' +
                             doc['destination.port'].value + ' : ' +
                             matcher.group(1);
                    }
                    return 'no_http_uri';"
        }
      }
    },
    "file_types": {
      "terms": {
        "script": {

```

```

        "source": "def uri = doc['http.uri'].value;
                    if (uri.contains('.exe')) return 'exe';
                    if (uri.contains('.ps1')) return 'powershell';
                    if (uri.contains('.msi')) return 'msi';
                    return 'other';"
    }
  }
},
"size": 0
}

```

F.4 LLM Analysis Output

When the aggregation results are retrieved and provided to the LLM, it corroborates across sources to answer forensic questions. The following output was generated by Claude Sonnet 4 (7-chunk context) for the question: *“What is the Windows user account name of the potentially infected machine in the LAN?”*

The LLM Analyzer correctly identified shutchenson as the Windows user account of the potentially infected machine. The model followed a structured three-step reasoning chain: first identifying the compromised IP (10.1.17.215) from Suricata alert patterns, then correlating it to hostname desktop-l8c5gsj via Kerberos hostname data, and finally disambiguating the human user account from machine accounts by explicitly filtering out accounts ending in \$. The model cited three source files in CITED_CHUNKS, selectively omitting four of the seven retrieved chunks as unused, demonstrating focused attribution across the cross-source reasoning chain (Listing 9).

Listing 9: LLM analysis demonstrating cross-query corroboration

3. What is the Windows human user account name (not machine accounts ending in \$) of the potentially infected machine in the LAN?

Looking at the security data, I need to identify the Windows human user account name of the potentially infected machine.

Analysis

Potentially Infected Machine Identification:

From the Suricata alerts, I can see several concerning security alerts including:

- "ET MALWARE Fake Microsoft Teams CnC Payload Request (GET)" (2 occurrences)
- "ET DROP Spamhaus DROP Listed Traffic Inbound group 5" (2 occurrences)
- "ET INFO PS1 Powershell File Request" (2 occurrences)

Machine and User Correlation:

The Kerberos data shows all authentication activity is coming from IP 10.1.17.215, which corresponds to hostname "desktop-l8c5gsj" based on the hostname aggregation data.

User Account Analysis:

From the Kerberos client accounts data for IP 10.1.17.215, I can see three accounts:

1. "shutchenson" - 11 authentication events (human user account)
2. "DESKTOP-L8C5GSJ\$" - 10 authentication events (machine account - ends in \$)
3. "desktop-l8c5gsj\$" - 4 authentication events (machine account - ends in \$)

Suspicious Activity Correlation:

The suspicious domains data shows that IP 10.1.17.215 is accessing multiple suspicious domains, and this same IP is generating malware alerts in Suricata. The machine accounts ending in "\$" are excluded per the question requirements, leaving "shutchenson" as the human user account.

Evidence Summary:

- Infected machine IP: 10.1.17.215
- Hostname: desktop-l8c5gsj
- Malware alerts: "ET MALWARE Fake Microsoft Teams CnC Payload Request"
- Human user account: shutchenson (11 Kerberos authentications from 10.1.17.215)

FINAL ANSWER = shutchenson

```

CITED_CHUNKS = [kerberos_clients_result.json, suricata_alerts_result.json, kerberos_hostnames_result.json]

RETRIEVED_CHUNKS = [kerberos_hostnames_result.json, suricata_alerts_result.json, suspicious_domains_result.json,
                    kerberos_clients_result.json, suspicious_certificates_result.json, kerberos_full_services_result.json,
                    suspicious_certificates_result.json]

```

G Comparative Analysis of LLM Reasoning

To illustrate how different LLMs approach security incident analysis, we compare outputs from three providers on the same question from the Koi Stealer scenario: “What are the suspicious external IP addresses contacted, which might be involved in command-and-control (C2) communication?” This question requires synthesizing evidence from multiple data sources (Suricata alerts, DNS logs, connection data) and distinguishing confirmed C2 infrastructure from merely suspicious traffic.

The reference answer for this scenario is 79.124.78.197 as the C2 server, which received 49 high-severity alerts including explicit Koi Stealer signatures.

G.1 DeepSeek V3: Precision-Focused Analysis

DeepSeek produces a structured analysis that systematically evaluates evidence strength and explicitly rejects weak indicators. Faced with multiple external IPs in the retrieved context, DeepSeek identifies 79.124.78.197 as the sole confirmed C2 server based on two convergent signals: 49 high-severity Suricata alerts with explicit Koi Stealer C2 signatures, and 218 connections in the high-activity external destination data. It explicitly considers other external IPs contacted by the victim host but rejects them individually, noting the absence of corroborating alert or signature data and even identifying benign infrastructure (Microsoft and Azure IPs) among the candidates. This demonstrates evidence-weighted reasoning: the model distinguishes between “contacted external IPs” and “confirmed C2 infrastructure,” prioritizing corroborated indicators over volume, and arrives at a single, high-confidence answer.

Listing 10: DeepSeek V3 analysis of C2 indicators

```

5. What are the suspicious external IP addresses contacted, which might be involved in command-and-control (C2)
   communication?

**Analysis**

The question asks for suspicious external IP addresses that might be involved in command-and-control (C2) communication. The
provided data shows network activity primarily from the internal host 172.17.0.99 (hostname desktop-rnvo9at, user
aflletcher).

Key evidence for identifying suspicious external IPs:

1. **Suricata Alerts:** The high_severity_alerts aggregation shows 49 alerts for the signature "ET HUNTING GENERIC
   SUSPICIOUS POST to Dotted Quad with Fake Browser 1". The destination for these alerts is the external IP 79.124.78.197.
   Furthermore, there are single alerts for "ET ATTACK_RESPONSE Koi Loader/Stealer CnC Config Inbound" and "ET MALWARE
   Win32/Koi Stealer CnC Checkin (GET)", which are explicitly named as C2-related. The source/destination data for these
   high-severity alerts points to communication between 172.17.0.99 and 79.124.78.197.

2. **High Activity Logs:** The high_activity_external_dest_result.json data strongly supports this, showing that internal
   host 172.17.0.99 contacted 35 unique external destinations. Its top destination by a massive margin (218 connections)
   is 79.124.78.197. This high volume of traffic to a single external IP, coupled with the C2-themed Suricata alerts to/
   from that same IP, is highly indicative of C2 communication.

3. **Other External IPs:** The same high-activity data lists other external IPs contacted by 172.17.0.99 (23.45.119.144,
   204.79.197.203, 52.113.194.132, 23.195.212.189). However, there is no direct alert or signature data linking these IPs
   to malicious activity. Some may be benign (e.g., 204.79.197.203 is a Microsoft IP, 52.113.194.132 is likely Azure).
   Without additional context like suspicious domains or alerts tied to them, they do not meet the threshold for "
   suspicious... involved in C2" based on the provided data.

**Conclusion:** The only external IP with direct, multiple pieces of evidence linking it to potential C2 activity is
79.124.78.197.

FINAL ANSWER = [79.124.78.197]

CITED_CHUNKS = [suricata_alerts_result.json, high_activity_external_dest_result.json]

```

```
RETRIEVED CHUNKS = [suspicious_domains_result.json, suricata_alerts_result.json, suricata_alerts_result.json,
kerberos_hostnames_result.json, kerberos_full_services_result.json, kerberos_clients_result.json,
high_activity_external_dest_result.json]
```

G.2 Cisco Foundation-Sec-8B: Verbose Chain-of-Thought

The security-specialized Cisco model produces verbose chain-of-thought reasoning that explicitly works through each data source before synthesizing a conclusion. Like DeepSeek, it correctly identifies 79.124.78.197 as the sole C2 server, but through a longer reasoning path that narrates its inspection of each retrieved chunk in sequence.

Listing 11: Cisco Foundation-Sec-8B analysis of C2 indicators (abbreviated)

```
The user is asking me to identify suspicious external IP addresses that might
be involved in Command-and-Control (C2) communication based on the provided
security data. I need to analyze the given JSON files and extract relevant
information.

First, I'll examine the `suricata_alerts_result.json` file under the
high_severity_alerts aggregation... Within this, there's a high_severity_sources
bucket. Looking inside it, I see an entry for `79.124.78.197` with a doc_count
of 49. The key associated with this IP is "ET HUNTING GENERIC SUSPICIOUS POST
to Dotted Quad with Fake Browser 1". This signature suggests potential malicious
activity, possibly related to C2...

[...sequential analysis of kerberos, high-activity destination, and domain data...]

Considering all the evidence, `79.124.78.197` stands out due to its association
with a high-severity Suricata alert signature. No other IP in the data provides
such clear and direct evidence of involvement in C2 communication.

Based on the provided security data, the following suspicious external IP
addresses were identified as potentially involved in C2 communication:

Suspicious External IP Addresses:
79.124.78.197

**FINAL ANSWER = [79.124.78.197]**

**CITED CHUNKS = [suricata_alerts_result.json]**

RETRIEVED CHUNKS = [suspicious_domains_result.json, suricata_alerts_result.json, suricata_alerts_result.json,
kerberos_hostnames_result.json, kerberos_full_services_result.json, kerberos_clients_result.json,
high_activity_external_dest_result.json]
```

Cisco arrives at the same correct answer as DeepSeek, but its reasoning is substantially more verbose: it narrates its inspection of each chunk sequentially rather than synthesizing across sources. The response cites only one chunk (`suricata_alerts_result.json`), whereas DeepSeek cites two, suggesting Cisco anchors on the strongest single signal rather than cross-validating. The output formatting is already atypical here: Cisco wraps the structured fields in Markdown bold (****** . . ******) rather than emitting plain text. Across the full evaluation, Cisco's formatting varied further—sometimes omitting brackets around answer lists, sometimes dropping CITED CHUNKS entirely—requiring manual review during scoring.

G.3 GPT-5.2: Enumerative Listing

GPT-5.2 produces a concise response that lists external IPs without differentiating evidence strength or justifying individual inclusions. The answer correctly identifies 79.124.78.197 but bundles it alongside four other IPs drawn from the high-activity destination data, including 204.79.197.203 and 52.113.194.132 which are Microsoft and Azure infrastructure respectively. No reasoning is provided for why these IPs are considered suspicious, nor does the model acknowledge the benign nature of known cloud provider addresses. Unlike DeepSeek, which explicitly evaluated and rejected these same IPs, GPT-5.2 treats the question as a data extraction task, listing all high-activity destinations without evaluating their relevance to the specific question of C2 communication.

Listing 12: GPT-5.2 analysis of C2 indicators

5. What are the suspicious external IP addresses contacted, which might be involved in command-and-control (C2) communication?

The suspicious external IP addresses contacted, which might be involved in command-and-control (C2) communication, are:

- 79.124.78.197
- 23.45.119.144
- 204.79.197.203
- 52.113.194.132
- 23.195.212.189

FINAL ANSWER = [79.124.78.197, 23.45.119.144, 204.79.197.203, 52.113.194.132, 23.195.212.189]

CITED CHUNKS = [suricata_alerts_result.json, high_activity_external_dest_result.json]

RETRIEVED CHUNKS = [suspicious_domains_result.json, suricata_alerts_result.json, suricata_alerts_result.json, kerberos_hostnames_result.json, kerberos_full_services_result.json, kerberos_clients_result.json, high_activity_external_dest_result.json]

GPT-5.2 conflates high-activity destination IPs with confirmed C2 infrastructure, reporting all external destinations above an activity threshold without evaluating their individual threat relevance. Including known Microsoft and Azure IPs (204.79.197.203, 52.113.194.132) as suspicious C2 candidates represents a failure to apply basic threat contextualization. The model treats the question as a data extraction task rather than an analytical one, reporting IPs present in the retrieved chunks without evaluating their relevance to the specific question of C2 communication.

H Precision-Recall Analysis

This appendix provides precision-recall metrics for set-valued questions across all 17 malware scenarios and 8 LLM providers. Set-valued questions are those whose reference answer contains more than one indicator, specifically questions about suspicious domains and URLs for initial infection, and suspicious external IPs for C2 communication.

- **Qs** denotes the number of set-valued questions in that scenario;
- **Ref** denotes the total number of reference indicators across those questions;
- **N** denotes the total number of indicators predicted by the model;
- **TP/N** shows precision as a ratio of true positives to predicted indicators;
- **Rec.** denotes mean recall averaged across set-valued questions within each scenario.

In cases where the model answers in sentence form rather than a delimiter-separated list (e.g., “45.131.214.85 is a C2 server”), **N** is adjusted to reflect the number of matched reference tokens rather than the number of parsed predicted items. The reference set captures *confirmed* malicious indicators from published solutions; several indicators reported as false positives exhibit genuinely suspicious characteristics, so true precision may be higher than reported.

The **Avg.** row reports aggregate precision as total TP / total N across all scenarios, and macro-averaged recall (mean of per-scenario recall values, consistent with the main evaluation).

Claude and DeepSeek achieve the highest aggregate precision (107/141 and 90/120 respectively) alongside strong recall, reflecting focused, evidence-grounded responses. Cisco and GPT models show substantially lower precision, driven by heavy over-prediction on scenarios such as fake_auth (Cisco: 6/51) and jun21isc (GPT models: 11–12/72).

I IcedID Dataset Complexity

Here we present some aspects that make the IcedID malware particularly challenging, and how our system uncovers attack indicators.

Multi-Host Environment with Mixed Traffic. The capture contains traffic from three Windows hosts in an Active Directory environment, with only one confirmed compromise. DESKTOP-SFF9LJF (10.4.19.136) follows the documented IcedID chain: Firebase-hosted installer, HTTP binary retrieval, persistent HTTPS C2 beaconing, and BackConnect activity. Other hosts in the environment (including DESKTOP-RETP4BU at 10.4.19.138) generate background traffic that includes DNS queries and web requests unrelated to the IcedID infection. This multi-host traffic creates analytical noise, as there is no structural boundary separating the infected host’s footprint from routine AD activity.

Volume as a Misleading Signal. Domain join sequences and routine AD operations from uninfected hosts produce heavy Kerberos and LDAP bursts that are volumetrically louder than the infected host’s quieter but diagnostically significant IcedID C2 beaconing, making raw traffic volume an unreliable primary indicator in this capture.

Reference Set Scope. The Unit 42 reference answers cover six IOCs for a single host. During evaluation, our system surfaced additional indicators confirmed valid upon manual investigation:

Table 9: Precision-Recall Analysis: Set-Valued Questions Across All Scenarios (single-answer questions excluded)

Dataset	Qs	Ref	Claude			DeepSeek			Ollama			Cisco			GPT-4			GPT-4o			GPT-5-mini			GPT-5.2		
			N	TP/N	Rec.	N	TP/N	Rec.	N	TP/N	Rec.	N	TP/N	Rec.	N	TP/N	Rec.	N	TP/N	Rec.	N	TP/N	Rec.	N	TP/N	Rec.
apr21isc	2	8	6	4/6	66.5	5	4/5	66.5	6	4/6	66.5	4	2/4	33.5	7	3/7	41.5	7	3/7	41.5	7	3/7	41.5	7	3/7	41.5
burnincandle	3	14	16	14/16	100.0	14	11/14	83.3	12	10/12	62.7	7	5/7	45.7	10	10/10	62.7	10	10/10	62.7	10	10/10	62.7	10	10/10	62.7
dec21isc	3	6	10	6/10	100.0	10	6/10	100.0	9	5/9	83.3	7	5/7	83.3	8	4/8	66.7	8	4/8	66.7	8	4/8	66.7	8	4/8	66.7
dirty_rat	2	6	4	3/4	62.5	5	1/5	12.5	3	0/3	0.0	4	0/4	0.0	11	1/11	25.0	11	1/11	25.0	11	1/11	25.0	11	1/11	25.0
easy_123	1	2	2	2/2	100.0	2	2/2	100.0	1	1/1	50.0	2	2/2	100.0	1	1/1	50.0	1	1/1	50.0	1	1/1	50.0	1	1/1	50.0
fake_auth	3	7	8	7/8	100.0	8	7/8	100.0	7	6/7	83.3	51	6/51	83.3	13	3/13	50.0	13	3/13	50.0	6	3/6	50.0	13	3/13	50.0
gozi	3	17	9	9/9	65.0	11	9/11	65.0	10	8/10	48.3	9	7/9	59.0	8	8/8	48.3	8	6/8	42.3	8	6/8	42.3	8	6/8	42.3
iced_id	2	6	7	4/7	75.0	6	3/6	62.5	5	3/5	50.0	5	3/5	50.0	32	3/32	50.0	32	3/32	50.0	32	3/32	50.0	32	3/32	50.0
jun21isc	6	21	29	20/29	97.2	16	14/16	73.7	22	12/22	59.8	12	6/12	34.7	72	11/72	55.5	72	12/72	63.8	73	11/73	55.5	72	11/72	55.5
koi_stealer	1	2	2	2/2	100.0	2	2/2	100.0	1	1/1	50.0	2	2/2	100.0	1	1/1	50.0	1	1/1	50.0	1	1/1	50.0	1	1/1	50.0
lumma_stealer	2	4	6	4/6	100.0	7	4/7	100.0	4	3/4	75.0	3	2/3	50.0	3	2/3	50.0	3	2/3	50.0	3	2/3	50.0	3	2/3	50.0
may21isc	1	2	2	2/2	100.0	2	2/2	100.0	1	1/1	50.0	2	1/2	50.0	1	1/1	50.0	1	1/1	50.0	1	1/1	50.0	1	1/1	50.0
net_rat	2	4	8	4/8	100.0	4	3/4	75.0	3	2/3	50.0	4	0/4	0.0	6	3/6	75.0	5	2/5	50.0	1	0/1	0.0	5	2/5	50.0
oct21isc	5	15	15	15/15	100.0	12	12/12	85.0	12	11/12	75.0	38	5/38	36.6	28	8/28	53.4	16	12/16	73.4	28	8/28	53.4	16	12/16	73.4
qakbot	2	7	6	3/6	60.0	4	3/4	60.0	5	2/5	35.0	11	1/11	25.0	16	1/16	25.0	7	0/7	0.0	16	1/16	25.0	16	0/16	0.0
redline_str	2	4	4	3/4	75.0	7	3/7	75.0	4	1/4	25.0	3	1/3	25.0	6	3/6	75.0	6	3/6	75.0	4	2/4	50.0	6	3/6	75.0
warmcookie	2	6	7	5/7	87.5	5	4/5	75.0	4	1/4	25.0	5	2/5	50.0	11	2/11	37.5	11	2/11	37.5	11	2/11	37.5	11	2/11	37.5
Avg.			—	107/141	87.6	—	90/120	78.4	—	71/109	52.3	—	50/169	48.6	—	65/234	50.9	—	66/212	49.3	—	59/221	44.7	—	65/221	48.8

- **Additional C2 domain.** Our system cited `spakernakurs.com` resolving to `104.168.53.18`, identical to `askamoshopsi.com`—and absent from the Unit 42 writeup. Manual lookup confirmed the domain resolves to shared IcedID infrastructure, consistent with a rotation or replacement C2 domain.
- **Beaconing regularity.** Our system’s classification of `skansnekssky.com` as persistent C2 activity prompted `tshark` verification, confirming ~302-second intervals with sub-second jitter across 69+ sessions, a pattern unquantified in the reference answers.

Query Adaptation. The IcedID installer delivery via Firebase Storage (`firebasestorage.googleapis.com`) required query modification: the standard `*googleapis.com*` exclusion for noise reduction would suppress the entire infection vector. We replaced broad wildcards with surgical per-subdomain exclusions and added two aggregations: (1) direct-IP TLS connections, recovering BackConnect C2 at `193.149.176.100`, and (2) HTTP requests to raw IP hosts, recovering the redirect server at `80.77.25.175`. This adaptation illustrates how attackers exploit common filtering heuristics and the need for dataset-aware query refinement.

J MITRE ATT&CK Technique Coverage

The two query libraries cover 16 of ~200 MITRE ATT&CK Enterprise techniques. Coverage is scoped by data source and attack type: our current sources are network PCAPs (via Zeek/Suricata) and Windows Event Logs. Broader coverage requires adding data sources and corresponding queries, and our RAG pipeline can be easily extended with additional sources and indicators. For example, endpoint EDR would cover process-level techniques (T1055 Process Injection, T1003 Credential Dumping), email gateway logs would cover initial delivery (T1566 Phishing), and proxy logs would cover web-based exfiltration (T1567 Exfiltration Over Web Service), etc. The malware-traffic library covers 6 techniques: T1071 C2 beaconing, T1573 Encrypted Channel, T1105 file downloads, T1568 suspicious domains, T1078 Kerberos auth, T1041 high-volume connections. Table 10 shows the AD attack query library with its MITRE ATT&CK mappings.

Table 10: Query Library for Active Directory Attack Detection

Query	MITRE ATT&CK
PowerShell Injection	T1059 – Command and Scripting Interpreter
File Drop	T1204 – User Execution
Certificate Enumeration	T1649 – Steal/Forge Authentication Certificates
Certificate Request	T1649 – Steal/Forge Authentication Certificates
Certificate Issued	T1649 – Steal/Forge Authentication Certificates
Kerberos Authentication	T1558 – Steal/Forge Kerberos Tickets
Kerberos Service Tickets	T1558 – Steal/Forge Kerberos Tickets
Service Installation	T1543 – Create/Modify System Process
User Account Creation	T1136 – Create Account
Security Group Modification	T1098 – Account Manipulation
SSH to SCADA Server	T1021.004 – Remote Services: SSH
SCADA Service Stopped	T1489 – Service Stop

K Choice of Analysis Window

The attack spans ~15 minutes; we split it into three 5-minute windows. A 5-minute window produces ~40 KB of extracted data, which fits in the prompt. The full 15-minute window produces ~100 KB, exceeding the prompt limit: Claude drops from 96% to 62%, with missed steps due to both data truncation and the model summarizing rather than enumerating steps in a longer context. Shorter windows (1–2 min) would fragment continuing activity across LLM calls.